

AI Personal Assistant Chatbot

SUBMITTED BY	ENVIRONMENT	MODEL ENDPOINT	EXECUTION STATUS
Mohammed Kaderji	Jupyter Notebook • Streamlit	Gemini 3.6 Flash	✓ Verified & Tested

1. Objective & Folder Structure

Objective: Build a responsive web-based personal assistant chatbot that answers arbitrary user questions using the Gemini API and Streamlit.

Folder Structure: AI_Chatbot/ → app.py, requirements.txt, README.md

2. Step-by-Step Implementation Guide

Step 1: Install Required Libraries

Open Jupyter Notebook, click into an empty code cell, enter the install command, and press **Shift + Enter**:

```
!pip install streamlit google-generativeai
```

Wait until the asterisk [*] turns into an execution number like [1].

Step 2: Create Project Folder & Files

Inside Jupyter, set up the dedicated workspace directory and files:

- Run `import os; os.makedirs("AI_Chatbot", exist_ok=True)`
- Run `%%writefile AI_Chatbot/requirements.txt` containing `streamlit` and `google-generativeai`.
- Run `%%writefile AI_Chatbot/README.md` with project description.

Step 3: Write app.py (Model Configuration & UI Logic)

Inside Jupyter, use `%%writefile AI_Chatbot/app.py` to create the core application script. Configure Gemini credentials, initialize the updated `gemini-3.6-flash` model to prevent 404 deprecation errors, construct the input field with `st.text_input()`, and bind inference to `st.button("Ask AI")`.

Step 4: Run Application (Server Launch)

Open Command Prompt (cmd), navigate to the folder, and run Streamlit on port 8501:

```
cd C:/Users/Hp/AI_Chatbot
```

```
streamlit run app.py --server.port 8501
```

Step 5: Verify Live Output

Open your browser at `http://localhost:8501`, enter the test question `"who is obama"`, and click **Ask AI** to verify structured generative response rendering.

3. Source Code Implementation (`app.py`)

```
import streamlit as st
import google.generativeai as genai

# 1. Configure Gemini API Credentials
genai.configure(api_key="YOUR_API_KEY")

# 2. Load Gemini model (targeted to gemini-3.6-flash)
model = genai.GenerativeModel("gemini-3.6-flash")

# 3. User Interface
st.title("🤖 AI Personal Assistant")
st.write("Ask me anything!")

question = st.text_input("Enter your question")

# 4. Process Question on Button Click
if st.button("Ask AI"):
    if question:
        response = model.generate_content(question)
        st.success(response.text)
    else:
        st.warning("Please enter a question.")
```

4. Execution Proof & Verification (localhost:8501)

Tested Live Parameters & Verification:

- **Target Host:** `http://localhost:8501` (Streamlit web application running locally).
- **Tested Query:** `"who is obama"`
- **Observed Output:** The assistant produced a detailed biographical breakdown structured into 3 formatted sections: (1) Key Accomplishments as President (Affordable Care Act, 2008 Economic Recovery, Foreign Policy, Social Equality, Nobel Peace Prize), (2) Early Life and Political Career (Harvard Law Review, Illinois State Senate), and (3) Life After the Presidency (Obama Foundation, Higher Ground Productions). Response was cleanly rendered in Streamlit's success alert banner.

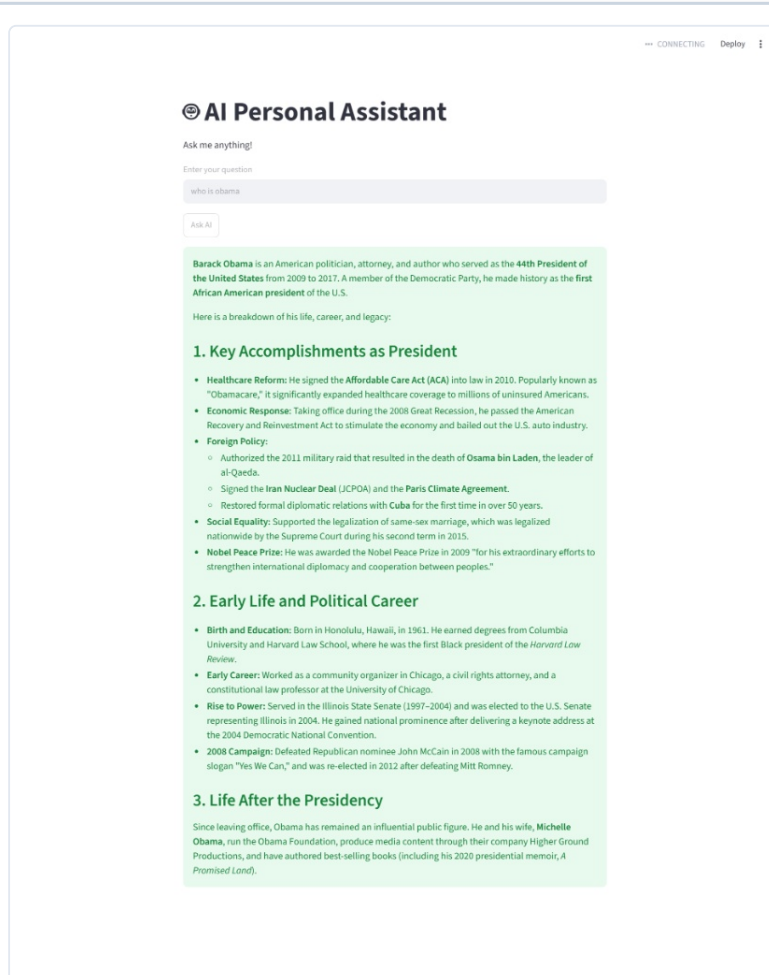


Figure 1.0: Real-time execution verification of AI Personal Assistant running on localhost:8501 with verified query output.